بخش اول

فصل ۱: معرفی سریعKafka

حجم داده‌های موجود در جهان به‌صورت **نمایی** در حال رشد است و طبق گزارش **مجمع جهانی اقتصاد**، تعداد بایت‌هایی که در جهان ذخیره شده‌اند اکنون بسیار بیشتر از تعداد ستاره‌های موجود در **جهان قابل مشاهده** است.

وقتی به این داده‌ها فکر می‌کنید، شاید تصور کنید که آن‌ها به شکل انبوهی از بایت‌ها در **انبارهای داده (Data Warehouse)**، **پایگاه‌های داده رابطه‌ای** یا **سیستم‌های فایل توزیع‌شده** قرار دارند. چنین سیستم‌هایی ما را عادت داده‌اند که داده را در **حالت سکون** تصور کنیم؛ یعنی داده جایی ذخیره شده و وقتی بخواهیم آن را پردازش کنیم، یک پرس‌وجو یا jobروی آن مجموعه داده اجرا می‌کنیم.

این دیدگاه، روش سنتی نگاه به داده است. اما در حالی که داده‌ها گاهی در جایی جمع می‌شوند، در بسیاری از موارد **در حال حرکت هستند**. بسیاری از سیستم‌ها جریان‌های پیوسته‌ای از داده تولید می‌کنند، از جمله:

- حسگرهای **IoT**
- حسگرهای پزشکی
- سیستم‌های مالی
- نرم‌افزارهای تحلیل کاربران و مشتریان
- لاگ‌های برنامه‌ها و سرورها
- و موارد دیگر

حتی داده‌هایی که در نهایت در جایی ذخیره می‌شوند نیز معمولاً پیش از رسیدن به محل نهایی خود **از طریق شبکه جابه‌جا می‌شوند**.

اگر بخواهیم داده‌ها را **در زمان واقعی (Real-Time)** و در حین حرکت پردازش کنیم، نمی‌توانیم منتظر بمانیم تا آن‌ها جایی جمع شوند و سپس در زمانی دلخواه یک پرس‌وجو یا job اجرا کنیم. این رویکرد برای برخی کاربردهای کسب‌وکار مناسب است، اما بسیاری از کاربردهای مهم نیاز دارند که داده‌ها **به‌صورت تدریجی و همزمان با رسیدنشان پردازش، غنی‌سازی، تبدیل و به آن‌ها واکنش داده شود**.

بنابراین به فناوری‌ای نیاز داریم که دیدگاه متفاوتی نسبت به داده داشته باشد: فناوری‌ای که به ما امکان دسترسی به داده‌ها در **حالت جریان** را بدهد و اجازه دهد با **جریان‌های پیوسته و نامحدود داده** به‌سرعت و با کارایی بالا کار کنیم.

اینجاست که **Apache Kafka**وارد می‌شود.

Apache Kafka (یا به‌اختصار **Kafka**) یک **پلتفرم جریان داده (Streaming Platform)** برای **دریافت، ذخیره‌سازی، دسترسی و پردازش جریان‌های داده** است. اگرچه کل این پلتفرم بسیار جالب است، اما تمرکز این کتاب روی بخشی است که از نظر من جذاب‌ترین قسمت Kafka است: **لایه پردازش جریان (Stream Processing Layer)**.

با این حال، برای درک **Kafka Streams** و **ksqlDB** (که هر دو در این لایه کار می‌کنند و ksqlDB همچنین در لایه دریافت داده نیز فعالیت دارد)، لازم است درک پایه‌ای از نحوه کار Kafka به‌عنوان یک پلتفرم داشته باشیم.

بنابراین در این فصل با برخی **مفاهیم و اصطلاحات مهم** آشنا می‌شوید که در ادامه کتاب به آن‌ها نیاز خواهید داشت. اگر قبلاً با Kafka آشنایی دارید، می‌توانید این فصل را رد کنید؛ در غیر این صورت، ادامه دهید.

در این فصل به پرسش‌های زیر پاسخ داده می‌شود:

- Kafkaچگونه ارتباط بین سیستم‌ها را ساده می‌کند؟
- اجزای اصلی معماری Kafka چیست؟
- کدام انتزاع ذخیره‌سازی بیشترین شباهت را به **جریان داده** دارد؟
- Kafkaچگونه داده‌ها را به شکلی **پایدار و مقاوم در برابر خطا** ذخیره می‌کند؟
- چگونه **دسترس‌پذیری بالا (High Availability)** و **تحمل خطا (Fault Tolerance)** در لایه‌های پردازش داده به دست می‌آید؟

در پایان این فصل، یک **آموزش عملی برای نصب و اجرای Kafka** ارائه خواهد شد. اما ابتدا بیایید **مدل ارتباطی Kafka**را بررسی کنیم.

مدل ارتباطی(Communication Model)

یکی از رایج‌ترین الگوهای ارتباط بین سیستم‌ها، **مدل همزمان کلاینت–سرور (Synchronous Client–Server)**است.

وقتی در این زمینه از «سیستم‌ها» صحبت می‌کنیم، منظور **برنامه‌ها، میکروسرویس‌ها، پایگاه‌های داده و هر چیزی است که از طریق شبکه داده می‌خواند یا می‌نویسد.**

مدل کلاینت–سرور در ابتدا ساده است و شامل **ارتباط مستقیم بین سیستم‌ها** می‌شود، همان‌طور که در شکل 1-1 نشان داده شده است.

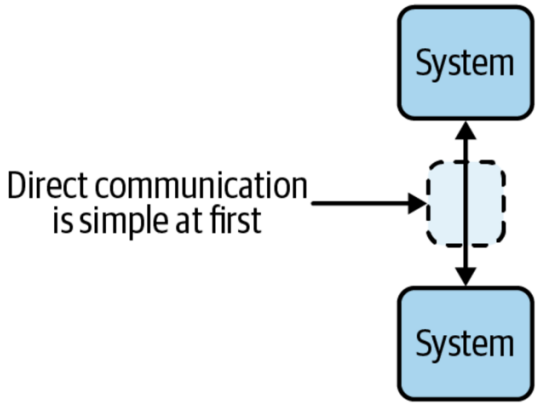


Figure 1-1. Point-to-point communication is simple to maintain and reason about when you have a small number of systems

شکل 1

برای مثال، ممکن است برنامه‌ای داشته باشید که به‌صورت **همزمان (synchronous)** برای دریافت داده از یک پایگاه داده پرس‌وجو ارسال می‌کند، یا مجموعه‌ای از **میکروسرویس‌ها** که مستقیماً با یکدیگر ارتباط برقرار می‌کنند.

اما وقتی تعداد سیستم‌هایی که باید با هم ارتباط داشته باشند بیشتر می‌شود، ارتباط **نقطه‌به‌نقطه (point-to-point)** به‌سختی قابل مقیاس‌پذیر شدن است. در نتیجه، شبکه‌ای پیچیده از مسیرهای ارتباطی به وجود می‌آید که **درک، مدیریت و نگهداری آن دشوار** می‌شود.

شکل 1-2 نشان می‌دهد که حتی با تعداد نسبتاً کمی از سیستم‌ها نیز این وضعیت تا چه حد می‌تواند **گیج‌کننده و پیچیده** شود.

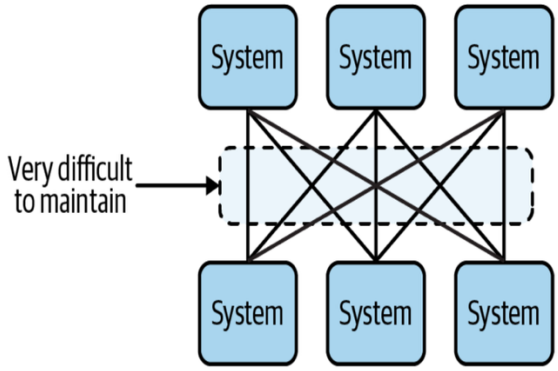


Figure 1-2. The result of adding more systems is a complex web of communication channels, which is difficult to maintain

شکل 2

برخی معایب مدل کلاینت–سرور

- **وابستگی زیاد سیستم‌ها :**سیستم‌ها به یکدیگر وابسته می‌شوند، زیرا ارتباطشان مستلزم شناخت متقابل است. این موضوع نگهداری و به‌روزرسانی سیستم‌ها را پیچیده‌تر از حد لازم می‌کند.
- **ارتباط همزمان محدودیت دارد :**ارتباط همزمان فضای کمی برای خطا باقی می‌گذارد، زیرا اگر یکی از سیستم‌ها آفلاین شود، تضمینی برای تحویل داده‌ها وجود ندارد.
- **تنوع پروتکل‌ها و استراتژی‌ها :**سیستم‌ها ممکن است از پروتکل‌های مختلف ارتباطی، استراتژی‌های مقیاس‌پذیری یا مدیریت خطا استفاده کنند. در نتیجه، ممکن است با گونه‌های مختلفی از سیستم‌ها روبه‌رو شویم **(Software Speciation)** که نگهداری را دشوار می‌کند و با اصل معروف "برنامه‌ها را مثل گاو نگه دارید نه حیوان خانگی" در تضاد است.
- **احتمال بارگذاری بیش از حد سیستم دریافت‌کننده :**سیستم‌های دریافت‌کننده کنترل سرعت ورود درخواست‌ها یا داده‌های جدید را ندارند. بدون یک بافر درخواست، آن‌ها باید طبق سرعت سیستم‌هایی که درخواست می‌فرستند عمل کنند.
- **تمرکز کم روی داده‌ها :**در مدل کلاینت–سرور، اصطلاحات بیشتر روی **درخواست و پاسخ** تأکید دارند و به **داده‌ها** به اندازه کافی توجه نمی‌شود. در سیستم‌های داده‌محور، داده باید محور اصلی باشد.
- **امکان بازپخش ارتباطات وجود ندارد :**این موضوع بازسازی وضعیت سیستم را دشوار می‌کند.

Kafka و مدل ارتباطی ساده‌تر

Kafkaارتباط بین سیستم‌ها را با عمل کردن به‌عنوان یک **هاب ارتباطی متمرکز** ساده می‌کند (که اغلب به آن مانند **سیستم عصبی مرکزی**گفته می‌شود). در این مدل، سیستم‌ها می‌توانند بدون نیاز به شناخت یکدیگر **داده ارسال و دریافت کنند.**

الگوی ارتباطی که Kafka پیاده می‌کند، **الگوی انتشار–اشتراک (Publish–Subscribe) یا (pub/sub)** است. نتیجه این الگو، یک **مدل ارتباطی بسیار ساده‌تر** است، همان‌طور که در شکل 1-3 نشان داده شده است.

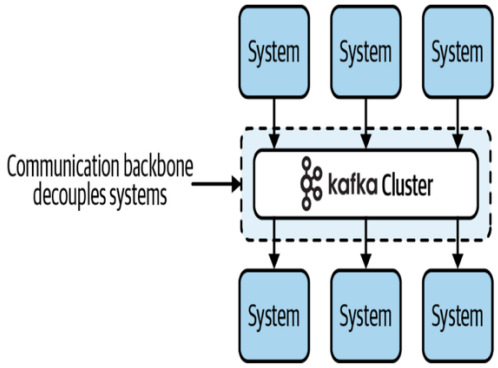


Figure 1-3. Kafka removes the complexity of point-to-point communication by acting as a communication hub between systems

شکل 3

اگر جزئیات بیشتری به نمودار قبلی اضافه کنیم، می‌توانیم **اجزای اصلی مدل ارتباطی Kafka** را شناسایی کنیم، همان‌طور که در شکل 1-4 نشان داده شده است.

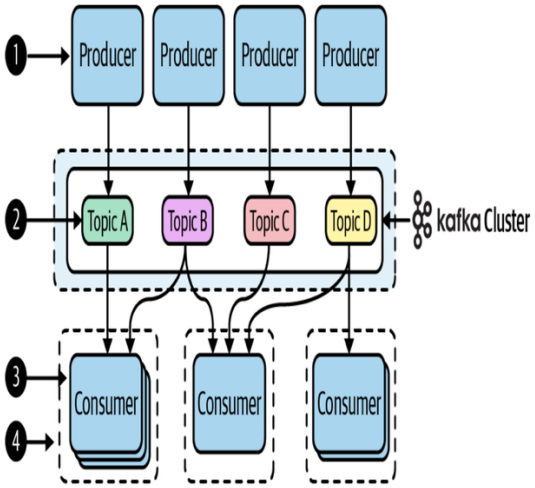


Figure 1-4. The Kafka communication model, redrawn with more detail to show the main components of the Kafka platform

شکل 4

- به جای اینکه سیستم‌های مختلف مستقیماً با یکدیگر ارتباط برقرار کنند، **تولیدکننده‌ها (Producers)** داده‌های خود را به یک یا چند **Topic**منتشر می‌کنند، بدون اینکه نگران باشند چه کسی آن داده‌ها را می‌خواند.
- **Topicها** جریان‌های داده‌ای با نام مشخص (یا کانال‌ها) هستند که داده‌های مرتبط در آن‌ها در یک **کلاستر Kafka**ذخیره می‌شوند. عملکرد آن‌ها شبیه **جدول‌ها در پایگاه داده** است؛ یعنی داده‌های مرتبط را گروه‌بندی

می‌کنند. اما برخلاف جدول‌ها، Topicها **اسکیما مشخصی ندارند** و داده‌ها به صورت بایت‌های خام ذخیره می‌شوند، که کار با آن‌ها را بسیار منعطف می‌کند.

- **مصرف‌کننده‌ها (Consumers)** فرآیندهایی هستند که داده‌های یک یا چند Topic را **می‌خوانند یا به آن‌ها مشترک می‌شوند .(subscribe)** آن‌ها مستقیماً با تولیدکننده‌ها ارتباط ندارند و فقط به داده‌های جریانی که به آن علاقه دارند گوش می‌دهند.

- مصرف‌کننده‌ها می‌توانند به صورت گروهی (**Consumer Group**) کار کنند تا **کار پردازشی بین چند فرآیند تقسیم شود** و عملکرد سیستم بهینه گردد.

مزایای مدل ارتباطیKafka

مدل ارتباطی Kafka که تأکید بیشتری بر **جریان‌های داده در حال حرکت** دارد و امکان **خواندن و نوشتن آسان توسط چندین فرآیند** را فراهم می‌کند، مزایای متعددی دارد:

- **کاهش وابستگی سیستم‌ها :**سیستم‌ها **جدا از هم (decoupled)** می‌شوند و نگهداری آن‌ها آسان‌تر است، زیرا می‌توانند داده تولید و مصرف کنند بدون اینکه نیاز به شناخت سایر سیستم‌ها داشته باشند.

- **ارتباط غیرهمزمان با تضمین تحویل قوی‌تر :**اگر یک مصرف‌کننده (Consumer) دچار مشکل شود، می‌تواند وقتی دوباره آنلاین شد، پردازش را **از همان نقطه‌ای که متوقف شده بود** ادامه دهد. یا اگر چند مصرف‌کننده در یک **گروه مصرف‌کننده (Consumer Group)** وجود داشته باشد، کار بین دیگر اعضا توزیع می‌شود.

- **استانداردسازی پروتکل و مقیاس‌پذیری :**سیستم‌ها می‌توانند از یک **پروتکل ارتباطی استاندارد** استفاده کنند Kafka)از یک **پروتکل TCP باینری با عملکرد بالا** استفاده می‌کند) و همچنین از **استراتژی‌های مقیاس‌پذیری و مکانیزم‌های تحمل خطا**)که توسطConsumer Group ها مدیریت می‌شوند) بهره ببرند. این موضوع امکان نوشتن نرم‌افزاری سازگار و قابل درک را فراهم می‌کند.

- **پردازش داده با سرعت قابل کنترل :**مصرف‌کننده‌ها می‌توانند داده‌ها را با سرعتی که می‌توانند پردازش کنند، مصرف کنند. داده‌های نخوانده تا زمانی که مصرف‌کننده آماده باشد، در Kafka ذخیره می‌شوند به‌صورت **پایدار و مقاوم در برابر خطا** .به عبارت دیگر، اگر جریان داده ناگهان به حجم بسیار بالا برسد، کلاستر Kafka نقش **بافر** را بازی می‌کند و از غرق شدن مصرف‌کننده‌ها جلوگیری می‌کند.

- **تمرکز روی داده به شکل رویدادها :**داده‌ها به شکل **رویداد (Event)** سازماندهی می‌شوند. هر رویداد قطعه‌ای از داده با یک ساختار مشخص است (که در بخش «Events» به آن می‌پردازیم). نکته اصلی این است که می‌توانیم تمرکز خود را روی داده‌های جریان‌دار قرار دهیم، نه صرفاً حل پیچیدگی لایه ارتباطی مانند مدل کلاینت–سرور.

- **امکان بازسازی وضعیت سیستم :**سیستم‌ها می‌توانند در هر زمان با **بازپخش رویدادها در یکTopic** ، وضعیت خود را بازسازی کنند.

یک تفاوت مهم بین **مدل انتشار–اشتراک (pub/sub)** و **مدل کلاینت–سرور** این است که در مدل pub/sub **ارتباط دوطرفه نیست؛** جریان‌ها یک‌طرفه هستند. به عبارت دیگر، اگر سیستمی داده‌ای را به یک Topic ارسال کند و انتظار داشته باشد سیستم دیگری آن را پردازش (مثلاً غنی‌سازی یا تبدیل) کند، داده‌ی پردازش‌شده باید در **Topicدیگری** نوشته شود تا سپس توسط فرآیند اولیه مصرف شود.

این روش ساده است، اما **دیدگاه ما نسبت به ارتباط را تغییر می‌دهد** .تا زمانی که به یاد داشته باشید **کانال‌های ارتباطی Topic)ها) ذاتاً جریان‌گونه هستند**)یک‌طرفه، با چند منبع و چند مصرف‌کننده(downstream ، طراحی سیستم‌ها آسان است: سیستم‌ها می‌توانند به جریان بایت‌های مورد علاقه خود گوش دهند و هر زمان که بخواهند داده‌ای را بهTopic ها ارسال کنند.

در فصل‌های بعدی، ما زیاد باTopic های Kafka کار خواهیم کرد (هر برنامه Kafka Streams و ksqlDB که می‌سازیم، معمولاً داده را از یک یا چند Topic می‌خواند و می‌نویسد)، بنابراین تا پایان کتاب این کار برای شما **طبیعی خواهد شد.**

حال که دیدیم مدل ارتباطی Kafka چگونه ارتباط بین سیستم‌ها را ساده می‌کند وTopic ها به‌عنوان **رسانه ارتباطی بین سیستم‌ها** عمل می‌کنند، بیایید نگاه عمیق‌تری به **نقش جریان‌ها در لایه ذخیره‌سازی Kafka** داشته باشیم.

جریان‌ها چگونه ذخیره می‌شوند؟

وقتی تیم مهندسی **LinkedIn**پتانسیل یک **پلتفرم داده‌محور جریان (stream-driven)** را دید، با یک سؤال مهم روبه‌رو شدند :**چگونه باید جریان‌های داده نامحدود و پیوسته را در لایه ذخیره‌سازی مدل‌سازی کرد؟**

در نهایت، **انتزاع ذخیره‌سازی (storage abstraction)** که شناسایی شد، قبلاً در بسیاری از سیستم‌های داده‌ای وجود داشت، از جمله:

- پایگاه‌های داده سنتی
- Key-value storeها
- سیستم‌های کنترل نسخه
- و غیره

این انتزاع چیزی نبود جز **Commit Logساده اما قدرتمند**، که در اینجا به آن فقط **Log**گفته می‌شود.

نکته: وقتی در این کتاب از **Log**صحبت می‌کنیم، منظورمان **لاگ‌های برنامه** نیستند (مانند لاگ‌های سرور HTTP که اطلاعاتی درباره اجرای یک برنامه منتشر می‌کنند). منظور، **یک ساختار داده‌ای مشخص** است که در پاراگراف‌های بعدی توضیح داده می‌شود.

Logها ساختار داده‌ای هستند که **فقط به آن اضافه می‌شود (append-only)** و یک **دنباله مرتب از رویدادها** را ثبت می‌کنند.

بیایید جزئیات این ویژگی‌ها را دقیق‌تر بررسی کنیم و با ایجاد یک **Logساده از خط فرمان**، با آن آشنا شویم. برای مثال، می‌توانیم یک Log به نام **user_purchases**ایجاد کنیم و با داده‌های نمونه آن را پر کنیم:

```
# create the logfile
touch users.log

# generate four dummy records in our log
echo "timestamp=1597373669,user_id=1,purchases=1" >> users.log
echo "timestamp=1597373669,user_id=2,purchases=1" >> users.log
echo "timestamp=1597373669,user_id=3,purchases=1" >> users.log
echo "timestamp=1597373669,user_id=4,purchases=1" >> users.log
```

حالا اگر به **Log**که ایجاد کردیم نگاه کنیم، می‌بینیم که شامل **چهار کاربر** است که هرکدام **یک خرید** انجام داده‌اند:

```
# print the contents of the log
cat users.log

# output
timestamp=1597373669,user_id=1,purchases=1
timestamp=1597373669,user_id=2,purchases=1
timestamp=1597373669,user_id=3,purchases=1
timestamp=1597373669,user_id=4,purchases=1
```

اولین ویژگی **Logها** این است که به صورت **append-only**نوشته می‌شوند. یعنی اگر **user_id=1**خرید دوم خود را انجام دهد، **سطر اول به‌روزرسانی نمی‌شود**، زیرا هر رکورد در یک Log **غیرقابل تغییر (immutable)** است.

در عوض، **رکورد جدید** به انتهای Log اضافه می‌شود:

```
# append a new record to the log
echo "timestamp=1597374265,user_id=1,purchases=2" >> users.log

# print the contents of the log
cat users.log

# output
timestamp=1597373669,user_id=1,purchases=1 ❶
timestamp=1597373669,user_id=2,purchases=1
timestamp=1597373669,user_id=3,purchases=1
timestamp=1597373669,user_id=4,purchases=1
timestamp=1597374265,user_id=1,purchases=2 ❷
```

- زمانی که یک رکورد در **Log**نوشته می‌شود، **غیرقابل تغییر (immutable)** در نظر گرفته می‌شود. بنابراین اگر نیاز به **به‌روزرسانی** داشته باشیم (مثلاً تغییر تعداد خرید یک کاربر)، **رکورد اصلی دست‌نخورده باقی می‌ماند**.
- برای مدل‌سازی به‌روزرسانی، کافی است **مقدار جدید را به انتهای Log اضافه کنیم**. در نتیجه، Logهم رکورد قدیمی و هم رکورد جدید را شامل می‌شود، و هر دو **غیرقابل تغییر** هستند.

هر سیستمی که بخواهد **تعداد خریدهای هر کاربر** را بررسی کند، می‌تواند به سادگی **هر رکورد را به ترتیب در Log بخواند**. آخرین رکورد برای **user_id=1**شامل مقدار به‌روزشده خواهد بود.

این موضوع ما را به **ویژگی دومLog ها** می‌رساند: **مرتب بودن.(ordered)**

Logنمونه قبلی به ترتیب **Timestamp**بود (ستون اول)، اما منظور ما از **مرتب بودن** صرفاً ترتیب زمانی نیست. در واقع، Kafkaبرای هر رکورد یک Timestamp ذخیره می‌کند، اما **لازم نیست رکوردها دقیقاً به ترتیب زمان ثبت شوند**.

وقتی می‌گوییم یک Log **مرتب است**، منظور این است که **موقعیت هر رکورد در Log ثابت است و هرگز تغییر نمی‌کند**.

اگر Log را دوباره چاپ کنیم و این بار **شماره سطرها** را اضافه کنیم، می‌توان موقعیت هر رکورد را در ستون اول مشاهده کرد:

```
# print the contents of the log, with line numbers
cat -n users.log

# output
1       timestamp=1597373669,user_id=1,purchases=1
2       timestamp=1597373669,user_id=2,purchases=1
3       timestamp=1597373669,user_id=3,purchases=1
4       timestamp=1597373669,user_id=4,purchases=1
5       timestamp=1597374265,user_id=1,purchases=2
```

حال تصور کنید سناریویی که **ترتیب رکوردها تضمین نشده باشد**. در این صورت، چندین فرآیند می‌توانند **به‌روزرسانی‌های user_id=1** را به ترتیب متفاوت بخوانند و در نتیجه درباره **تعداد واقعی خریدهای این کاربر** اختلاف نظر پیش آید. با اطمینان از **مرتب بودنLog ها**، داده‌ها می‌توانند به‌صورت **قطعی (deterministic)** توسط چندین فرآیند پردازش شوند.

علاوه بر این، در حالی که موقعیت هر رکورد در مثال قبلی با **شماره سطر** مشخص شده بود، Kafkaاز اصطلاح **offset**برای موقعیت هر رکورد در Log توزیع‌شده خود استفاده می‌کند.

- **Offset**از 0 شروع می‌شود و امکان رفتار مهمی را فراهم می‌کند: هر **گروه مصرف‌کننده (Consumer Group)** می‌تواند **به‌طور مستقل از همان Log داده‌ها را بخواند** و **موقعیت خود در جریان Log/**را نگه دارد. این موضوع در شکل 1-5 نشان داده شده است.

حالا که با **لایه ذخیره‌سازی مبتنی بر Log در Kafka** آشنا شدیم و یک Log ساده از خط فرمان ایجاد کردیم، می‌توانیم این مفاهیم را به **ساختارهای سطح بالاتر در مدل ارتباطی Kafka** پیوند دهیم.

در ادامه، بحث **Topicها** را ادامه می‌دهیم و با مفهومی به نام **Partition**آشنا می‌شویم.

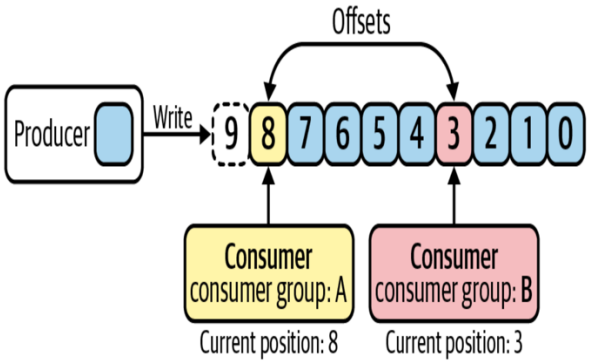


Figure 1-5. Multiple consumer groups can read from the same log, each maintaining their position based on the offset they have read/processed

شکل 5

Topicها و Partition ها

در بحث **مدل ارتباطی Kafka** یاد گرفتیم که Kafka دارای **جریان‌های نام‌گذاری‌شده‌ای به نام Topic** است.

علاوه بر این، **Topicهای Kafka** بسیار **انعطاف‌پذیر** هستند و می‌توانند داده‌های مختلفی را در خود نگه دارند:

- می‌توان **Topicهای همگن (homogeneous)** داشت که فقط شامل **یک نوع داده** هستند.
- یا **Topicهای ناهمگن (heterogeneous)** که شامل **چند نوع داده** می‌باشند.

یک تصویر از این استراتژی‌های مختلف در شکل 1-6 نشان داده شده است.

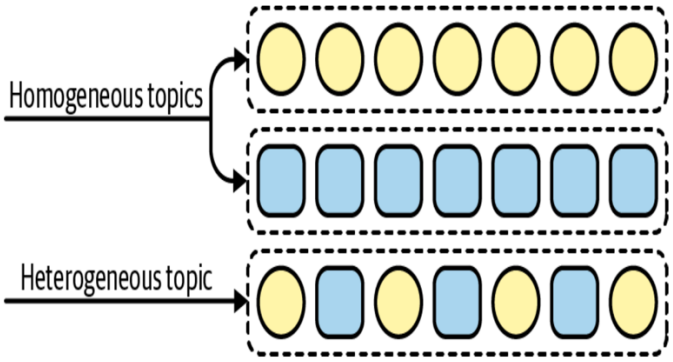


Figure 1-6. Different strategies exist for storing events in topics; homogeneous topics generally contain one event type (e.g., `clicks`) while heterogeneous topics contain multiple event types (e.g., `clicks` and `page_views`)

شکل 6

ما همچنین یاد گرفتیم که **Logهای فقط افزودنی (append-only commit logs)** برای مدل‌سازی جریان‌ها در لایه ذخیره‌سازی Kafka استفاده می‌شوند.

آیا این بدان معناست که **هر Topic معادل یک فایل Log** است؟ نه دقیقاً.

Kafkaیک **Logتوزیع‌شده (distributed log)** است و توزیع کردن تنها یک Log کار سختی است. بنابراین اگر بخواهیم **سطحی از موازی‌سازی (parallelism)** در توزیع و پردازشLog ها داشته باشیم، نیاز است که **چندین Log ایجاد کنیم.**

به همین دلیل، **Topicهای Kafka به واحدهای کوچک‌تری به نام Partition تقسیم می‌شوند.**

- **Partitionها** همانLog های جداگانه هستند (یعنی همان ساختار داده‌هایی که در بخش قبل توضیح داده شد) که داده‌ها از آن‌ها تولید و مصرف می‌شوند.
- از آنجا که **انتزاع commit log** در سطح Partition پیاده‌سازی شده است، **ترتیب رکوردها تضمین‌شده است** و هر Partition **مجموعه‌ای ازoffset ها** مخصوص به خود دارد.
- **ترتیب جهانی در سطح Topic پشتیبانی نمی‌شود،** به همین دلیل تولیدکننده‌ها معمولاً **رکوردهای مرتبط را به یک Partition مشخص می‌فرستند.**

به طور ایده‌آل، داده‌ها به صورت **تقریباً یکنواخت** بین تمامPartition های یک Topic توزیع می‌شوند، اما ممکن است **Partitionهایی با اندازه‌های متفاوت** نیز وجود داشته باشند.

شکل 1-7 یک مثال از Topic با سه Partition مختلف را نشان می‌دهد.

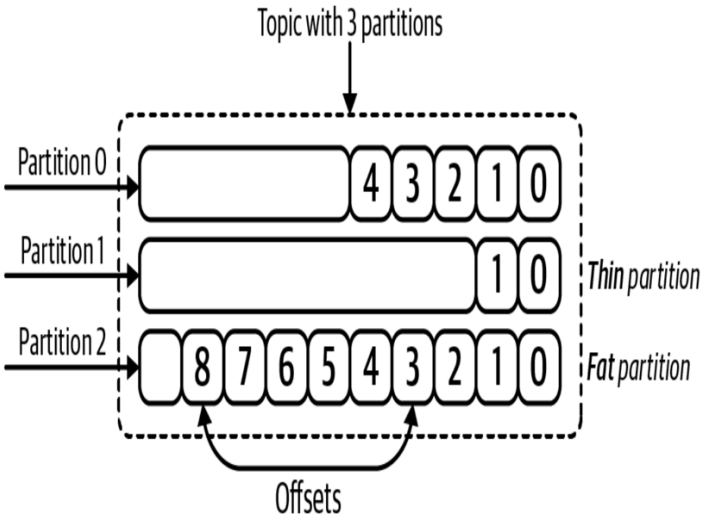


Figure 1-7. A Kafka topic configured with three partitions

شکل 7

تعداد **Partitionها** برای هر Topic قابل تنظیم است و داشتن **Partitionهای بیشتر** معمولاً به معنای **افزایش موازی‌سازی و توان عملیاتی (throughput)** است، هرچند **تعداد زیادPartition ها معایبی هم دارد** که در طول کتاب به آن‌ها می‌پردازیم.

نکته مهم این است که **در هر گروه مصرف‌کننده (Consumer Group) تنها یک مصرف‌کننده می‌تواند از یک Partitionداده بخواند** .با این حال، **اعضای گروه‌های مختلف می‌توانند از همان Partition داده بخوانند**، همان‌طور که در شکل 1-5 نشان داده شد.

- بنابراین اگر بخواهید بار پردازشی را بین **Nمصرف‌کننده در یک گروه** تقسیم کنید، به **N Partition**نیاز دارید.
- اگر تعداد اعضای گروه کمتر از تعدادPartition های Topic منبع باشد، مشکلی نیست؛ هر مصرف‌کننده می‌تواند چند Partition را پردازش کند.
- اگر تعداد اعضای گروه بیشتر ازPartition های Topic منبع باشد، برخی مصرف‌کننده‌ها بلااستفاده خواهند بود.

با توجه به این نکات، می‌توان تعریف **Topic**را بهبود داد: **Topic**یک جریان نام‌گذاری‌شده است که از چند Partition تشکیل شده است. و هر Partition **به عنوان یک Commit Logمدل می‌شود** که داده‌ها را به صورت **مرتب و فقط افزودنی (append-only)** ذخیره می‌کند.

رویدادها(Events)

در این کتاب، بخش زیادی از زمان خود را به **پردازش داده‌ها درTopic ها** اختصاص می‌دهیم. با این حال، هنوز **درک کاملی از نوع داده‌های ذخیره‌شده در Topic و به‌طور خاص درPartition های آن** پیدا نکرده‌ایم.

در ادبیات موجود دربارهKafka ، از اصطلاحات مختلفی برای توصیف داده‌ها در Topic استفاده می‌شود، از جمله:

- **Message**

- **Record**
- **Event**

این اصطلاحات اغلب به جای یکدیگر استفاده می‌شوند، اما در این کتاب، ما عمدتاً از **Event** استفاده می‌کنیم.

Event یک **جفت کلید–مقدار (key-value)** با **Timestamp** است که یک اتفاق یا رخداد را ثبت می‌کند.

آناتومی پایه‌ای هر Event که در یک **Partition Topic** ثبت می‌شود، در شکل 1-8 نشان داده شده است.

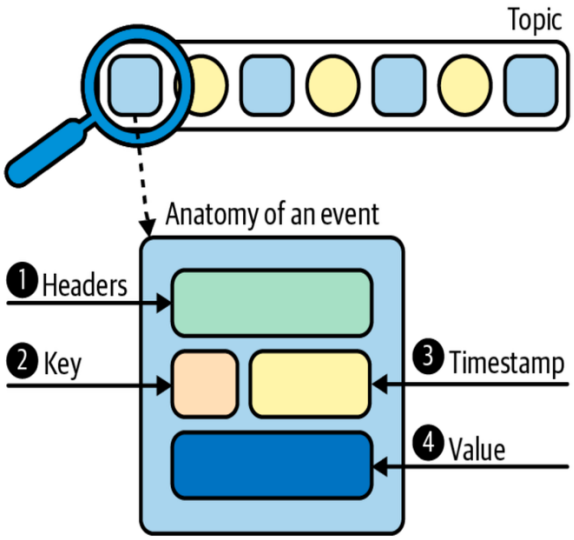


Figure 1-8. Anatomy of an event, which is what is stored in topic partitions

شکل 8

- **Headers** در **سطح برنامه** شامل متادیتای اختیاری درباره یک **Event** هستند. در این کتاب، زیاد با آن‌ها کار نمی‌کنیم.
- **Key‌ها** نیز اختیاری هستند، اما در **توزیع داده‌ها بین Partition ها** نقش مهمی دارند. در فصل‌های بعدی جزئیات بیشتری خواهیم دید، اما به‌طور کلی، Key‌ها برای **شناسایی رکوردهای مرتبط** استفاده می‌شوند.
- هر Event با یک **Timestamp** مرتبط است. جزئیات بیشتر درباره Timestamp ها در **فصل 5** توضیح داده می‌شود.
- **Value** شامل **محتوای اصلی پیام** است که به صورت **آرایه‌ای از بایت‌ها (byte array)** ذخیره می‌شود. مسئولیت تبدیل بایت‌های خام به یک ساختار معنی‌دارتر (مثلاً یک شی JSON یا یک رکورد Avro) بر عهده کلاینت‌ها است. جزئیات مربوط به **Serialization/Deserialization** در فصل مربوطه بررسی خواهد شد.

حالا که **درک خوبی از نوع داده‌های ذخیره‌شده در Topic** داریم، بیایید نگاه عمیق‌تری به **مدل استقرار کلاستری Kafka** داشته باشیم. این موضوع اطلاعات بیشتری درباره **نحوه ذخیره فیزیکی داده‌ها در Kafka** در اختیار ما قرار می‌دهد.

کلاستر Kafka و Broker ها

داشـــتن یـک **نقطـه ارتبـاط مرکزی** بـه این معنی اســـت کـه **قـابلیت اطمینـان و تحمـل خطـا** اهمیت زیـادی پیدا می‌کند. همچنین، **زیرساخت ارتباطی** باید **قابل مقیاس** باشد، یعنی بتواند حجم بالای بار کاری را مدیریت کند.

به همین دلیل، **Kafkaبه‌صورت یک کلاستر عمل می‌کند** و چندین ماشین، که به آن‌ها **Broker**گفته می‌شود، در **ذخیره و بازیابی داده‌ها**نقش دارند.

- کلاسـترهای Kafka می‌توانند بسـیار بزرگ باشـند و حتی **چند مرکز داده و مناطق جغرافیایی مختلف** را پوشـش دهند.
- در این کتاب، معمولاً با **کلاســـتر Kafka تـکـنود** کار خواهیم کرد، زیرا برای شـــروع اســـتفاده از **Kafka Streams**و **ksqlDB**کافی است.
- در محیط تولید(Production) ، معمولاً حداقل **ســـه Broker** نیاز دارید و باید **Replication Topic**ها را تنظیم کنید تا داده‌ها **بین چند Broker تکثیر شـــوند** .این کار باعث **دســـترســـی بالا (High Availability)** و جلوگیری از **از دست رفتن داده‌ها** در صورت خرابی یک ماشین می‌شود.

وقتی درباره **ذخیره و تکثیر داده‌ها درBroker ها** صـــحبت می‌کنیم، در واقع منظورمان **Partition**های **جداگانه یک Topic**است. برای مثال، یک Topic ممکن است **سه Partition** داشته باشد که بین **سه Broker مختلف** توزیع شده‌اند، همان‌طور که در **شکل 1-9** نشان داده شده است.

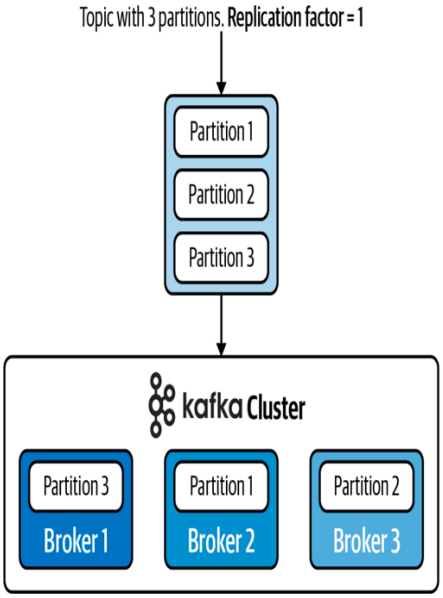


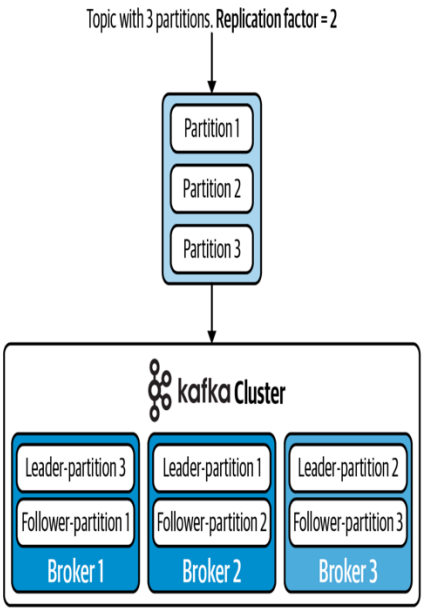
Figure 1-9. Partitions are spread across the available brokers, meaning that a topic can span multiple machines in the Kafka cluster

شکل 9

همان‌طور که مشـاهده می‌کنید، این قابلیت باعث می‌شود که **Topicها بتوانند بسـیار بزرگ شـوند**، زیرا می‌توانند فراتر از ظرفیت یک ماشین رشد کنند.

برای **تحمل خطا و دسترسی بالا (fault tolerance** و**high availability)** ، می‌توان **Replication Factor**را هنگام پیکربندی Topic تعیین کرد.

به عنوان مثال، **Replication Factorبرابر با 2** به این معنی است که هر Partition **روی دو Broker مختلف ذخیره می‌شود**. این موضوع در **شکل 1-10** نشان داده شده است.



شکل 10

هرگاه یک **Partition**روی چند Broker تکثیر می‌شود، یکی از Brokerها به عنوان **Leader**تعیین می‌شود.

- **Leader**مسئول **پردازش همه درخواست‌های خواندن و نوشتن** از تولیدکننده‌ها و مصرف‌کننده‌ها برای آن Partitionاست.
- سایر Brokerهایی که Partition تکثیرشده را دارند، به عنوان **Follower**شناخته می‌شوند و **داده‌ها را از Leaderکپی می‌کنند**.
- اگر Leader دچار مشکل شود، یکی از Followerها **به Leader جدید ارتقا می‌یابد**.

علاوه بر این، با افزایش بار کاری کلاستر، می‌توان **با افزودنBroker های جدید و انجام Reassignment Partitionها**، کلاستر را گسترش داد. این کار اجازه می‌دهد داده‌ها از ماشین‌های قدیمی به ماشین جدید منتقل شوند.

همچنین، Brokerها نقش مهمی در **مدیریت عضویت گروه‌های مصرف‌کننده (Consumer Groups)** دارند که در بخش بعدی توضیح داده می‌شود.

گروه‌های مصرف‌کننده(Consumer Groups)

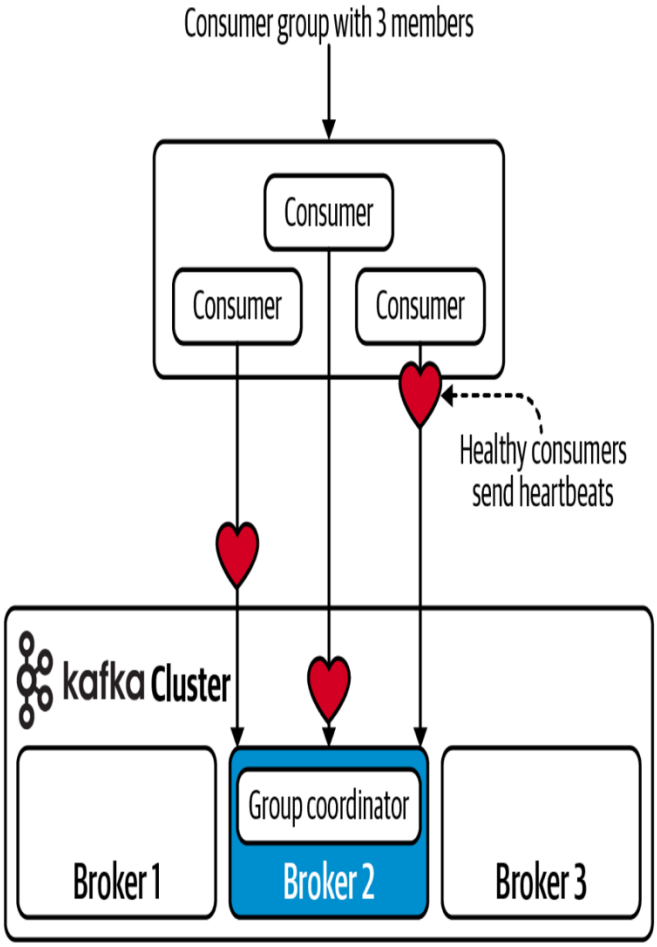
Kafkaبرای **توان عملیاتی بالا و تأخیر کم** بهینه‌سازی شده است. برای بهره‌برداری از این قابلیت در سمت مصرف‌کننده، نیاز داریم **کارها به‌صورت موازی بین چند فرآیند تقسیم شوند**. این کار با **گروه‌های مصرف‌کننده (Consumer Groups)**انجام می‌شود.

- **Consumer Group**شامل چند **مصرف‌کننده همکار** است و **عضویت در گروه می‌تواند در طول زمان تغییر کند.**
  - مصرف‌کننده‌های جدید می‌توانند اضافه شوند تا بار پردازشی افزایش یابد.
  - مصرف‌کننده‌ها ممکن است به صورت برنامه‌ریزی‌شده یا به دلیل خطای غیرمنتظره آفلاین شوند.
- Kafkaنیاز دارد که **عضویت هر گروه را مدیریت کرده و در صورت نیاز، کارها را مجدداً توزیع کند.**

برای این کار، هر Consumer Group به یک **Broker**ویژه به نام **Group Coordinator** اختصاص داده می‌شود:

- **Group Coordinator**مسئول **دریافت Heartbeat ها از مصرف‌کننده‌ها** و **شروع Rebalance کارها** هرگاه مصرف‌کننده‌ای مرده تشخیص داده شود.

شکل 1-11 نشان می‌دهد که مصرف‌کننده‌ها چگونه Heartbeat خود را به Group Coordinator ارسال می‌کنند.



شکل 11

هر عضو فعال در **گروه مصرف‌کننده** می‌تواند **یک یا چند Partition** را دریافت کند .

به عنوان مثال، توزیع کار بین **سه مصرف‌کننده سالم** می‌تواند همانند **شکل 1-12** باشد.

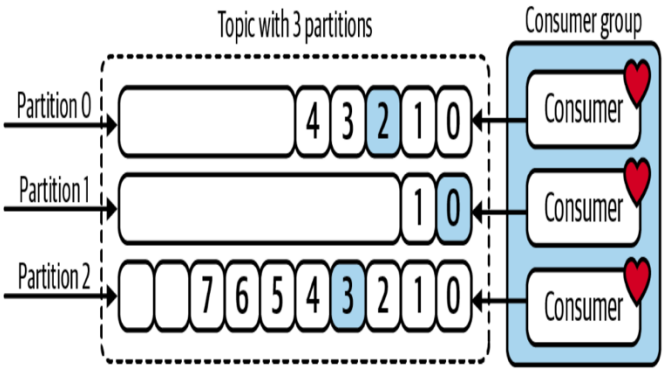


Figure 1-12. Three healthy consumers splitting the read/processing workload of a three-partition Kafka topic

شکل 12

با این حال، اگر یک **مصرف‌کننده (Consumer) دچار مشکل شود** و نتواند Heartbeat خود را به کلاستر ارسال کند، **کارها به‌طور خودکار به مصرف‌کننده‌های سالم منتقل می‌شوند.**

به عنوان مثال، در **شکل 1-13**، **مصرف‌کننده وسط** Partitionی را دریافت کرده است که قبلاً توسط مصرف‌کننده ناسالم پردازش می‌شد.

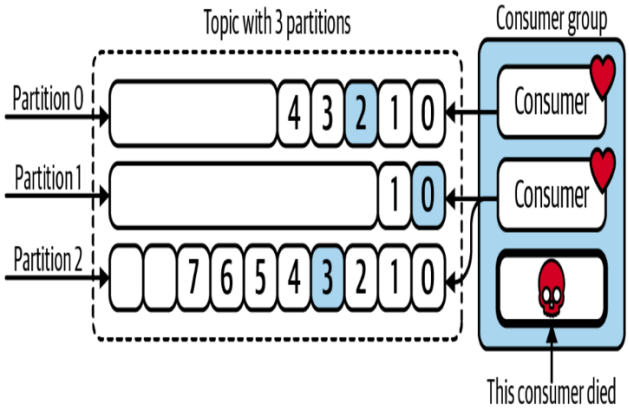


Figure 1-13. Work is redistributed when consumer processes fail

شکل 13

همان‌طور که مشاهده می‌کنید، **گروه‌های مصرف‌کننده** نقش بسیار مهمی در **دستیابی به دسترسی بالا و تحمل خطا در لایه پردازش داده** دارند.

با این مقدمه، حالا آموزش نصب Kafka را آغاز می‌کنیم.

نصبKafka

در **مستندات رسمیKafka** ، دستورالعمل‌های مفصل برای نصب دستی وجود دارد. اما برای ساده‌تر شدن کار، بیشتر آموزش‌های این کتاب از **Docker**استفاده می‌کنند. Docker اجازه می‌دهد **Kafkaو برنامه‌های پردازش جریان (stream processing)**را در یک محیط **کانتینریزه** اجرا کنیم.

بنابراین، در این آموزش **Kafkaبا Docker Compose** نصب خواهد شد و از **Docker Imageهای منتشرشده توسطConfluent** استفاده می‌کنیم.

مرحله اول :دانلود و نصب Docker از صفحه رسمی نصب.Docker

سپس، پیکربندی زیر را در فایلی به نام **docker-compose.yml**ذخیره کنید:

```yaml
---
version: '2'

services:
  zookeeper: ❶
    image: confluentinc/cp-zookeeper:6.0.0
    hostname: zookeeper
    container_name: zookeeper
    ports:
      - "2181:2181"
    environment:
      ZOOKEEPER_CLIENT_PORT: 2181
      ZOOKEEPER_TICK_TIME: 2000

  kafka: ❷
    image: confluentinc/cp-enterprise-kafka:6.0.0
    hostname: kafka
    container_name: kafka
    depends_on:
      - zookeeper
    ports:
      - "29092:29092"
    environment:
      KAFKA_BROKER_ID: 1
      KAFKA_ZOOKEEPER_CONNECT: 'zookeeper:2181'
      KAFKA_LISTENER_SECURITY_PROTOCOL_MAP: |
         PLAINTEXT:PLAINTEXT,PLAINTEXT_HOST:PLAINTEXT
      KAFKA_ADVERTISED_LISTENERS: |
         PLAINTEXT://kafka:9092,PLAINTEXT_HOST://localhost:29092
      KAFKA_OFFSETS_TOPIC_REPLICATION_FACTOR: 1
      KAFKA_TRANSACTION_STATE_LOG_REPLICATION_FACTOR: 1
```

شکل 14

کانتینر اول که نام آن را **zookeeper**گذاشته‌ایم، شامل نصب **ZooKeeper**خواهد بود. در این مقدمه درباره ZooKeeperصحبت نکردیم، زیرا در زمان نگارش این کتاب، **به‌تدریج از Kafka حذف می‌شود** .با این حال، ZooKeeperیک **سرویس متمرکز برای ذخیره متادیتا** مثل پیکربندیTopic ها است. هنوز در این آموزش آورده شده است، زیرا کتاب قبل از حذف کامل ZooKeeper منتشر شده است.

کانتینر دوم که نام آن **kafka**است، شامل نصب **Kafka**خواهد بود. در این کانتینر، **Broker**ما (که شامل کلاستر تک‌نود است) اجرا می‌شود و همچنین برخی از **اسکریپت‌های کنسول Kafka** برای تعامل با کلاستر را اجرا خواهیم کرد.

برای شروع کلاستر Kafka محلی، دستور زیر را اجرا کنید:

```
docker-compose up
```

با راه‌اندازی کلاسترKafka ، آماده‌ایم تا آموزش را ادامه دهیم.

سلام، Kafka

در این آموزش ساده، نشان خواهیم داد چگونه:

1. یک **Topic** ایجاد کنیم،
2. با استفاده از **Producer** داده به Topic بنویسیم، و
3. با استفاده از **Consumer** داده‌ها را از Topic بخوانیم.

ابتدا باید وارد کانتینری شویم که Kafka در آن نصب شده است. برای این کار دستور زیر را اجرا کنید:

```
docker-compose exec kafka bash
```

حال، یک Topic به نام **users** ایجاد می‌کنیم. برای این کار از یکی از اسکریپت‌های کنسول Kafka به نام **kafka-topics** استفاده خواهیم کرد. دستور ایجاد Topic به صورت زیر است:

```
kafka-topics \
--bootstrap-server localhost:9092 \
--create \
--topic users \
--partitions 4 \
--replication-factor 1# output
Created topic users.
```

kafka-topics یک اسکریپت کنسول است که همراه با Kafka ارائه می‌شود.

- **Bootstrap server**، جفت host/IP برای یک یا چند Broker است.
- این اسکریپت گزینه‌های متعددی برای کار با Topic ها دارد، از جمله:
  - --list
  - --describe
  - --delete
- در اینجا از **گزینه --create** استفاده می‌کنیم، زیرا قصد داریم یک Topic جدید ایجاد کنیم.

تنظیمات Topic:

- نام Topic: **users**
- تعداد Partition ها **4** :
- Replication Factor: **1** (چون در حال اجرای کلاستر تک‌نود هستیم). در محیط تولید، معمولاً این مقدار را بالاتر (مثلاً 3) قرار می‌دهید تا **دسترسی بالا** تضمین شود.

توجه: اسکریپت‌های کنسول در این بخش، همراه با توزیع اصلی Kafka ارائه می‌شوند. در نصب معمولی Kafka ، این اسکریپت‌ها دارای پسوند .sh هستند (مثل kafka-topics.sh، kafka-console-producer.sh و غیره). اما در **Confluent Platform** پسوند فایل حذف شده است، به همین دلیل در مثال قبل از kafka-topics استفاده کردیم، نه kafka-topics.sh.

پس از ایجاد Topic ، می‌توانید **توضیحات و پیکربندی Topic** را با دستور زیر مشاهده کنید:

```
kafka-topics \
--bootstrap-server localhost:9092 \
--describe \
--topic users
# output
```

```
Topic: users    PartitionCount: 4       ReplicationFactor: 1
Configs:
        Topic: users    Partition: 0    Leader: 1       Replicas: 1
Isr: 1
        Topic: users    Partition: 1    Leader: 1       Replicas: 1
Isr: 1
        Topic: users    Partition: 2    Leader: 1       Replicas: 1
Isr: 1
        Topic: users    Partition: 3    Leader: 1       Replicas: 1
Isr: 1
```

- گزینه **describe--**به ما اجازه می‌دهد **اطلاعات پیکربندی یک Topic مشخص** را مشاهده کنیم.

حال، بیایید با استفاده از اسکریپت **kafka-console-producer**داده‌هایی تولید کنیم:

```
kafka-console-producer \
--bootstrap-server localhost:9092 \
--property key.separator=, \
--property parse.key=true \
--topic users
```

- اسکریپت **kafka-console-producer**که همراه با Kafka ارائه می‌شود، برای **ارسال داده به یک Topic**استفاده می‌شود .با این حال، زمانی که با **Kafka Streams**و **ksqlDB**کار کنیم، **فرآیندهای Producer**در **کتابخانه Java جاسازی می‌شوند** و دیگر نیازی به استفاده از این اسکریپت در محیط‌های واقعی نداریم؛ فقط برای **تست و توسعه** کاربرد دارد.
- ما مجموعه‌ای از **جفت‌های کلید-مقدار (key-value)** را به **users** Topic ارسال خواهیم کرد.در این مثال، **کلید و مقدار با کاراکتر ,جدا می‌شوند.**

دستور قبلی شما را به **یک محیط تعاملی (interactive prompt)** می‌برد. در این محیط می‌توانید چندین جفت **key-value**به Topic users ارسال کنید.

پس از پایان، با فشار دادن **Control-C**از محیط خارج شوید.

```
>1,mitch
>2,elyse
>3,isabelle
>4,sammy
```

پس از ارسال داده‌ها بهTopic ، می‌توانیم با استفاده از اسکریپت **kafka-console-consumer**داده‌ها را بخوانیم.

دستور زیر نحوه انجام این کار را نشان می‌دهد:

```
kafka-console-consumer \
--bootstrap-server localhost:9092 \
```

```
--topic users \
--from-beginning
# output
mitch
elyse
isabelle
sammy
```

اسکریپت **kafka-console-consumer**نیز همراه با توزیع Kafka ارائه می‌شود.

مشابه اسکریپت **kafka-console-producer**، بیشتر آموزش‌های این کتاب **فرآیندهای Consumer تعبیه‌شده در Kafka Streams**و **ksqlDB** را استفاده می‌کنند و از این اسکریپت مستقل فقط برای **تست و توسعه** بهره می‌برند.

گزینه **--from-beginning**مشخص می‌کند که **خواندن داده‌ها از ابتدای Topic** شروع شود.

به صورت پیش‌فرض، **kafka-console-consumerفقط مقدار پیام (value) را نمایش می‌دهد** .اما همان‌طور که قبلاً یاد گرفتیم، هر Event شامل اطلاعات بیشتری است، از جمله:

کلید(Key)

زمان‌برچسب(Timestamp)

هدرها(Headers)

برای مشاهده **کلید وTimestamp** ، می‌توانیم **برخی ویژگی‌های اضافی** را به Console Consumer اضافه کنیم.

```
kafka-console-consumer \
--bootstrap-server localhost:9092 \--topic users \
--property print.timestamp=true \
--property print.key=true \
--property print.value=true \
--from-beginning
# output
CreateTime:1598226962606 1 mitch
CreateTime:1598226964342 2 elyse
CreateTime:1598226966732 3 isabelle
CreateTime:1598226968731 4 sammy
```

همین است! اکنون شما یاد گرفته‌اید چگونه با یک **کلاستر Kafka** تعاملات پایه‌ای انجام دهید.

مرحله نهایی، **خاموش کردن کلاستر محلی** است. برای این کار دستور زیر را اجرا کنید:

```
docker-compose down
```

جمع‌بندی

مدل ارتباطی Kafka ارتباط بین چندین سیستم را ساده می‌کند و **لایه ذخیره‌سازی سریع، پایدار و تنها افزودنی (append-only)**آن امکان کار با جریان‌های داده سریع را به راحتی فراهم می‌کند.

با استفاده از **پیاده‌سازی کلاستری**، Kafkaمی‌تواند در **لایه ذخیره‌سازی** با **تکثیر داده‌ها روی چندین ماشین (broker)**دسترسی بالا و تحمل خطا را تضمین کند.

علاوه بر این، توانایی کلاستر در دریافت **heartbeat از فرآیندهای مصرف‌کننده** و به‌روزرسانی **عضویت گروه‌های مصرف‌کننده** باعث می‌شود که در **لایه پردازش و مصرف جریان‌ها** نیز دسترسی بالا، تحمل خطا و **مقیاس‌پذیری بار کاری** حاصل شود.

تمام این ویژگی‌ها Kafka را به یکی از **محبوب‌ترین پلتفرم‌های پردازش جریان داده** تبدیل کرده است.

اکنون شما دانش کافی برای شروع کار با **Kafka Streams** و **ksqlDB** دارید. در بخش بعد، سفر خود را با Kafka Streams آغاز خواهیم کرد، با بررسی جایگاه آن در **اکوسیستم گسترده‌تر Kafka** و یادگیری نحوه استفاده از این کتابخانه برای کار با داده‌ها در **لایه پردازش جریان**.

توضیحات و یادداشت‌ها

1. در فصل ۳ درباره **آرایه‌های بایت خام** ذخیره‌شده در Topic ها و فرآیند **Deserialization** آن‌ها به ساختارهای سطح بالاتر مثل **JSON یا Avro** صحبت خواهیم کرد.
2. توسعه اولیه Kafka توسط **Jay Kreps**، **Neha Narkhede** و **Jun Rao** هدایت شد.
3. **Deterministic** یعنی همان ورودی‌ها همیشه همان خروجی‌ها را تولید می‌کنند.
4. به همین دلیل، پایگاه‌های داده سنتی از **Log‌ها برای تکثیر داده‌ها (Replication)** استفاده می‌کنند. Log‌ها هر عملیات نوشتن روی دیتابیس اصلی را ثبت کرده و همان عملیات را به همان ترتیب روی دیتابیس Replica انجام می‌دهند تا مجموعه داده مشابهی روی ماشین دیگر به صورت قطعی بازسازی شود.
5. مقاله جالبی از **Martin Kleppmann** در این موضوع وجود دارد: لینک مقاله. این مقاله درباره **مزایا و معایب استراتژی‌های مختلف** و دلایل انتخاب هر روش صحبت می‌کند. همچنین مقاله پیگیری **Robert Yokota** جزئیات بیشتری درباره پشتیبانی از انواع Event های مختلف با استفاده از **Confluent Schema Registry** ارائه می‌دهد.
6. **استراتژی پارتیشن‌بندی** قابل پیکربندی است، اما یک استراتژی رایج (که در Kafka Streams و ksqlDB هم استفاده می‌شود) این است که **پارتیشن بر اساس Key رکورد** تعیین شود (که می‌تواند از payload رکورد استخراج شود یا صریحاً تنظیم گردد). جزئیات بیشتر در فصل‌های بعد ارائه خواهد شد.
7. معایب داشتن تعداد زیاد پارتیشن شامل: **طولانی شدن زمان بازیابی بعد از برخی خطاها، افزایش مصرف منابع (File Descriptor، حافظه) و افزایش تأخیر End-to-End** است.
8. برای اجرای Kafka چندین **Docker Image** وجود دارد. تصاویر **Confluent** گزینه مناسبی هستند، چون Confluent همچنین Docker Image هایی برای فناوری‌های دیگر کتاب مثل **ksqlDB** و **Confluent Schema Registry** ارائه می‌دهد.
9. از نسخه ۲.۷ می‌توان از **--property print.headers=true** برای نمایش **Headers پیام‌ها** نیز استفاده کرد.